AReaL-DTA: Dynamic Tree Attention for Efficient Reinforcement Learning of Large Language Models

Up to **8.31**× training throughput by *not* computing the same prefix twice

Jiarui Zhang

May 1, 2026

AReaL Team, Ant Group
jiarui-z23@mails.tsinghua.edu.cn

Outline

Motivation: long shared prefixes recomputed in every forward & backward

RL Post-training Is Powerful — and Painfully Expensive

RL has become the engine driving the next wave of LLM capabilities:

- Math reasoning, code generation
- Multi-turn agentic tasks
- Long chain-of-thought training

But on the systems side, the bill is huge:

- **Compute heavy (training)**: long context's activations
- **Memory heavy (rollout)**: long context's KV cache
- **Throughput balance**: need to balance training and rollout in asynchronous scenarios

The hidden waste

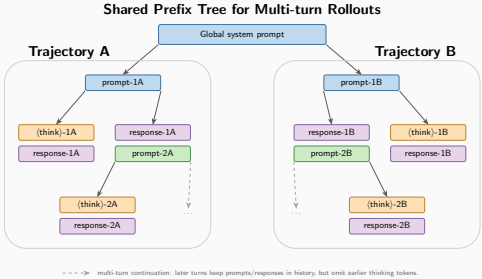
Rollouts for the same prompt share **long prefixes** — yet today's frameworks can recompute them in *every forward and every backward pass*.

Can we compute the shared prefix only once?

Where Does the Sharing Come From?

In multi-turn agentic RL (τ^2-Bench)

- Long **global system prompt** ($\sim$ 4.8K tokens, identical for all rollouts)
- Multi-turn dialogue history is shared across continuations, except at points where earlier-turn *thinking* tokens are dropped—these are the divergence points



Multi-turn rollouts naturally form a **prefix tree**.

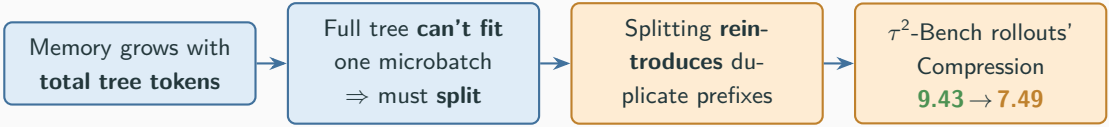
Compression rate on raw τ^2-Bench rollouts:

9.43$\times$ $\iff$ **89.4**% prefix sharing

Why Not Just Use Tree-Mask Attention?

Static Tree-Mask Attention Doesn't Scale to Training

Tree attention works for inference (i.e., speculative decoding). In *training*, the chain of consequences kills the gain:



Weakness of gradient checkpointing: it is layer-wise, not token-wise — so it cannot exploit tree-structured, shared-prefix rollouts for KV reuse.

Hardware-side penalty: FlexAttention (i.e., a publicly available tree-mask baseline)'s general mask kernel under-utilizes Hopper GPUs vs. FlashAttention.

*We need memory bounded by the **longest path**, not the whole tree.*

AReaL-DTA: Dynamic Tree Attention

Key Idea: Walk the Tree — Don't Materialize It

Traditional training: one full forward → one full backward, per microbatch.

AReaL-DTA breaks this contract:

- Build a **prefix tree** of all rollouts.
- Traverse it with **depth-first search (DFS)**.
- Interleave *forwards* with *backwards*.

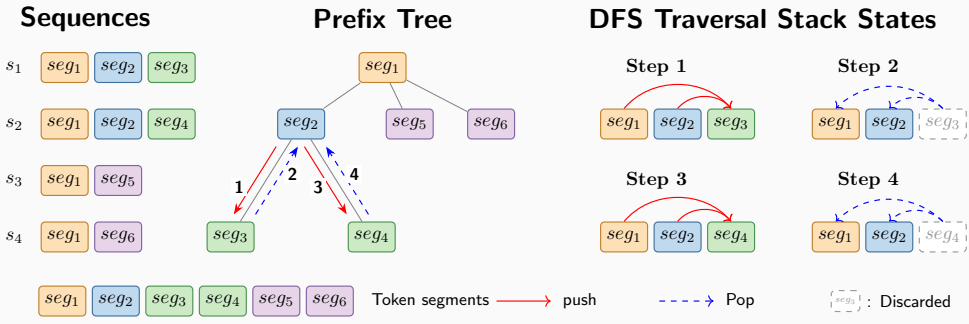
Memory is bounded by the **longest root-to-leaf path**, not the total token count of the tree.

Push Walk down a branch; reuse cached prefix KV; forward only the new token segment and push it onto the stack.

Visit At each leaf: compute the loss and trigger backward immediately.

Pop Backpropagation accumulates gradients in shared prefix nodes; release activations from the last stack token segment, but keep the prefix state for the next push (i.e., it's sibling branch).

DFS Traversal: One Path at a Time



Shared prefix is computed **once** and reused across siblings.

Why This Saves Memory — and Compute

Space complexity

Naive tree-mask $\mathcal{O}$(total tree tokens)

AReaL-DTA $\mathcal{O}$(longest path)

$\Rightarrow$ no need to fully materialize attention masks or activations of the whole tree.

Compute reuse

- Each shared-prefix node is forwarded **once**.
- Each leaf's loss gradient is backpropagated immediately.
- Prefix gradients are correctly *accumulated* from all descendants.

Live computation graph = only the path we are currently descending.

System Optimizations

Chunked backpropagation

- Long sequences ($>$ chunk_size) split into chunks.
- Recompute chunk activations using stored prefix KV cache.
- At `chunk_size = 4096`: extra-forward overhead $< \mathbf{10}\%$
  (vs. $\sim 25\%$ for gradient checkpointing).

Cut Tail (CT)

Skip KV-cache for *leaf* nodes' token segments — they're never reused as recompute anchors.

Optimal DFS order

Balance the segment lengths *per backward pass* to avoid frequent, inefficient short backward steps.

Larger backward chunks (LB)

Given enough memory space, larger chunks further amortize the extra-forward cost.

Scaling Out: Load-balanced Dispatch

Load-balanced Multi-GPU Dispatch

Problem. Split N rollouts into K trees, one per trainer GPU,
minimize the slowest GPU's cost

$$\mathcal{C} = \min \max_{j=1\ldots K} \mathcal{C}(\mathcal{T}_j)$$

Two competing goals:

- Balance per-GPU work
- Preserve prefix sharing maximally

AReaL-DTA's recipe

1. Sort rollouts in DFS order, i.e., lexicographic order $\Rightarrow$ similar prefixes are adjacent.
2. Cut the ordered list into K *contiguous* segments.
3. Binary-search the smallest threshold τ that admits a K-cut with $\mathcal{C}(\mathcal{T}_j) \leq \tau$.

Disabling this DP balancer costs **11.93%** system performance.

Evaluation: up to $8.31\times$ training throughput

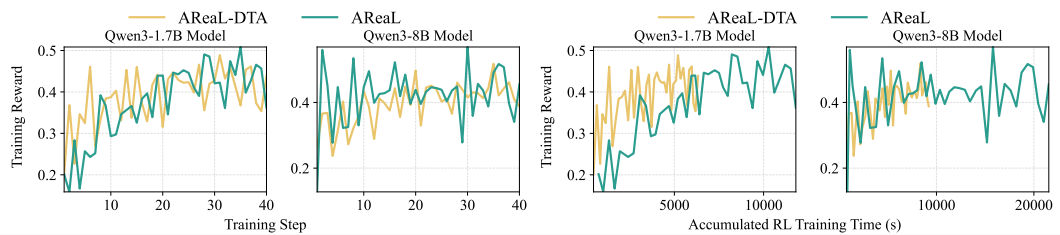
Setup

- **Workload:** τ^2-Bench multi-turn agentic RL.
- **Algorithm:** PPO; reference AReaL (state-of-the-art async RL system).
- **Models:** Qwen-3 family (1.7B, 4B, 8B, 14B).
- **Hardware:** 8× NVIDIA H200 GPUs.
- **Context length:** 16K (activation checkpointing on for AReaL and chunked-backpropagation on for AReaL-DTA).

Model	System	Parallel Strategy (rollout + train)
1.7B	AReaL-DTA	dp5-pp1-tp1 + dp3-pp1-tp1
	AReaL	dp2-pp1-tp1 + dp6-pp1-tp1
8B	AReaL-DTA	dp5-pp1-tp1 + dp3-pp1-tp1
	AReaL	dp2-pp1-tp1 + dp6-pp1-tp1

Optimal asynchronous RL training configuration for the parallel strategy.

End-to-End: Similar Reward, Much Faster



Reward vs. **training step**

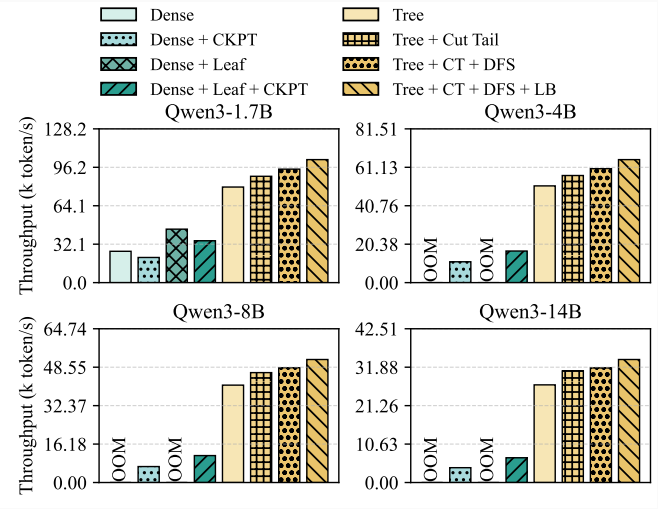
Reward vs. **wall-clock training time**

End-to-end throughput: Qwen3-1.7B: **1.28**× / Qwen3-8B: **2.28**×

Stability is preserved — AReaL-DTA does not degrade async RL training.

Under asynchronous settings, with the improved model training performance, AReaL-DTA will allocate more GPUs to the rollout generation phase for the optimal end-to-end throughput.

Training-Side Throughput: Up to 8.31×

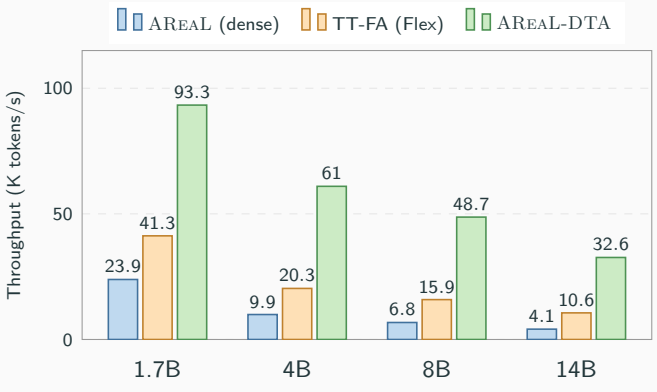


Compared to dense baseline with activation checkpointing:

- Vanilla Tree: 6.59×
- + Cut Tail: 7.53×
- + DFS order: 7.74×
- + Larger backward chunk: 8.31×

Extra forward overhead kept $< \mathbf{10\%}$ of total training time.

Head-to-Head vs. FlexAttention Tree-Mask (TT-FA)



Single-trainer throughput on τ^2-Bench (K tokens/s).

Speedup over AReaL

TT-FA: 1.73× → 2.58×
AReaL-DTA: **3.91**× → **7.94**×

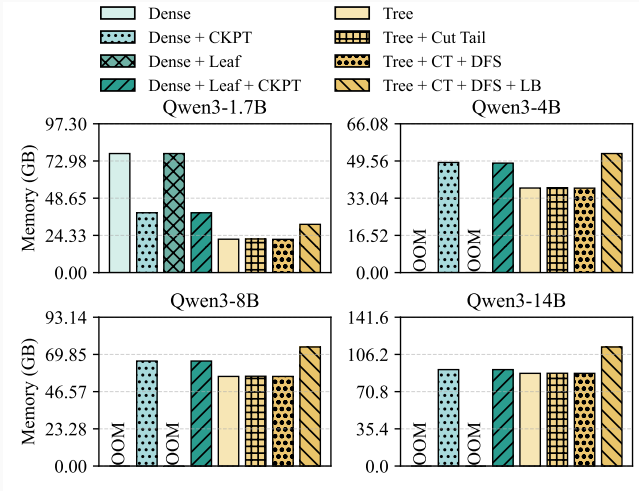
AReaL-DTA vs. TT-FA

2.26× → 3.08× (gap grows with model size)

Three sources of the gap:

- No tree splitting ⇒ *full* prefix sharing preserved.
- Chunked backprop on cached prefix KV: extra-forward $<$ **10%** (TT-FA still uses gradient ckpt $\sim$ **25%**).
- FlashAttention-3 kernel > FlexAttention MFU on Hopper.

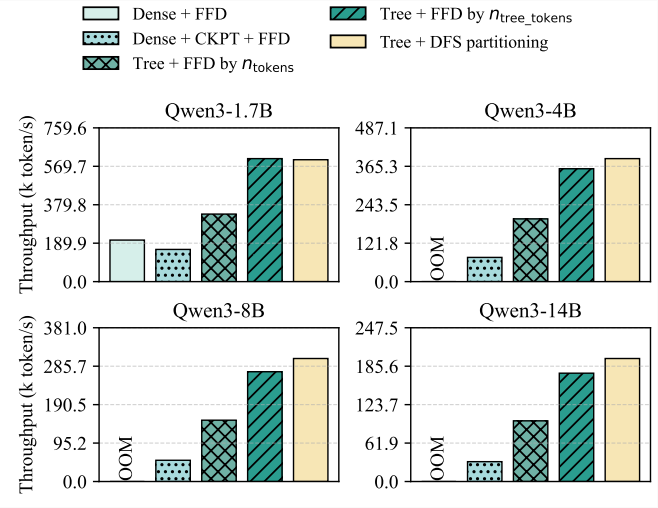
Memory: AReaL-DTA Removes the Need for Activation Checkpointing



- Dense baselines OOM on 4B / 8B / 14B without CKPT.
- Tree variants stay within budget; peak memory cut $>$ **50%** on large models.

How? AReaL-DTA adopts *chunked backpropagation*: it recomputes chunk activations from cached prefix KV, which acts as *token-wise* gradient checkpointing and leverages tree-structured rollouts for efficient KV reuse.

Multi-GPU Scaling: Throughput Holds Up to 8 GPUs



Backward throughput on 8 GPUs (Qwen3 1.7B / 4B / 8B / 14B).

- Validated at 2 / 4 / 8 GPUs — same trend.
- DFS-aware partition keeps per-GPU work even *and* prefix reuse high.

Removing the load balancer:
—**11.93%** training performance.

Takeaways

Why AReaL-DTA Matters

- RL post-training has hidden **tree-shaped** structure — treating it as a flat list of sequences wastes compute and memory.
- Static tree-mask attention struggles to take full advantage of prefix sharing, since splitting microbatches can break shared segments.
- AReaL-DTA **dynamically** carries out tree-structured attention for shared-prefix rollouts: **depth-first traversal** of the prefix tree walks one path at a time, interleaving forwards and backwards (bounded memory by longest path).

Rethinking *how attention is executed* for RL workloads is a promising direction for the next generation of LLM training systems.

AReaL-DTA: Open-Source Branch & Configuration

Paper: arxiv.org/abs/2602.00482

Code (development branch with DTA, to be merged):

github.com/inclusionAI/AReaL (feat/dta)

DTA + DFS-aware packing: For both actor **and** critic, set:

- tree_training_mode: dta
- packing_algorithm: dta

Example YAML (τ^2-Bench):

examples/tau2/dta

Thank you!

AReaL-DTA

Questions?